Vector Space Retrieval Model

1. Introduction

- **Vector Space Retrieval Model**
  - A statistical retrieval model
  - Designed to overcome key limitations of the Boolean Model
  - One of the most influential models in Information Retrieval
- **Learning objectives:**
  - Understand ranked retrieval
  - Understand vector representation of documents and queries
  - Understand TF–IDF and cosine similarity

Motivation: From Boolean to Ranked Retrieval

Why do we need another retrieval model?

- Boolean retrieval:
  - Exact matching only
  - Binary relevance (true / false)
  - No ranking of results
- Real users:
  - Focus on top results
  - Expect documents to be ranked by relevance
- **Vector Space Model answers:**
  "How relevant is a document to the query?"

Core Idea of the Vector Space Model

- **Key idea**
  - Each term corresponds to a dimension
  - Documents are represented as vectors
  - The query is also represented as a vector
  - Relevance is measured by similarity between vectors
- Retrieval is no longer exact matching
  Retrieval becomes graded relevance

Vector Space Representation

- After preprocessing, assume t distinct terms remain
- These terms form the vocabulary
- Each term defines one orthogonal axis

Dimensionality:

Dimensionality = t = |vocabulary|

- Each term, i, in a document or query, j, is given a real-valued weight, w_{ij}.
- Both documents and queries are expressed as t-dimensional vectors:

$$d_j = (w_{1j}, w_{2j}, \ldots, w_{tj})$$

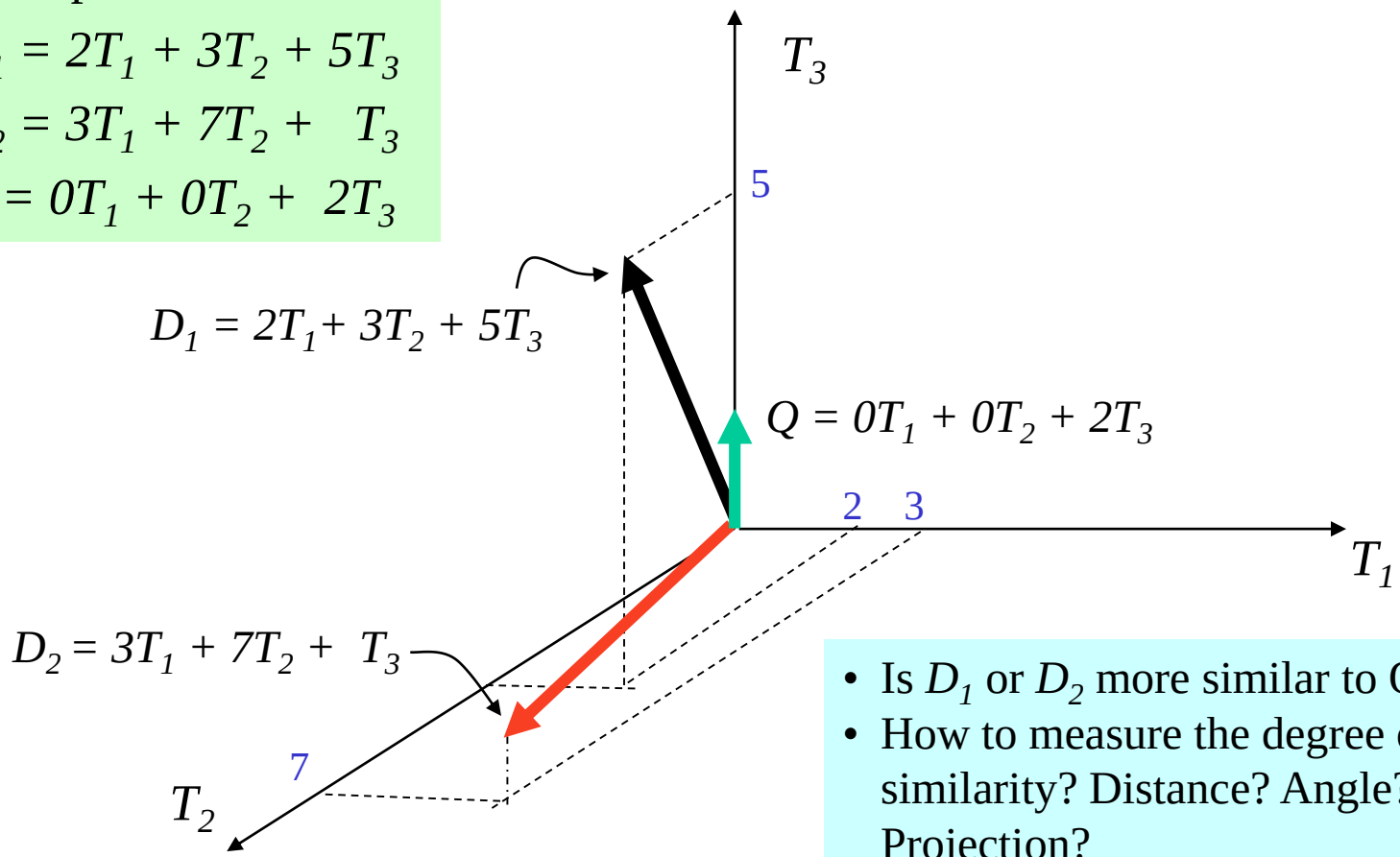
Graphic Representation

Example:

$D_1 = 2T_1 + 3T_2 + 5T_3$

$D_2 = 3T_1 + 7T_2 + T_3$

$Q = 0T_1 + 0T_2 + 2T_3$



- Is D_1 or D_2 more similar to Q?
- How to measure the degree of similarity? Distance? Angle? Projection?

Document Collection

- **Term–Document Matrix**
  - A collection of documents is represented as a matrix
  - Rows: terms
  - Columns: documents
  - Each entry represents the weight of a term in a document
- A zero value means:
  - The term does not appear
  - Or has no significance in the document

$$
\begin{pmatrix}
 & T_1 & T_2 & \ldots & T_t \\
D_1 & w_{11} & w_{21} & \ldots & w_{t1} \\
D_2 & w_{12} & w_{22} & \ldots & w_{t2} \\
\vdots & \vdots & \vdots & & \vdots \\
\vdots & \vdots & \vdots & & \vdots \\
D_n & w_{1n} & w_{2n} & \ldots & w_{tn}
\end{pmatrix}
$$

Why Term Weighting Is Needed

- Some terms occur frequently but are not informative
- Some terms are rare but highly discriminative

=> We need a principled way to assign weights to terms

Term Weights: Term Frequency

- Terms that occur more frequently in a document are assumed to be more important.

  f_{ij} = frequency of term i in document j

- Normalized term frequency:

  $tf_{ij} = f_{ij} / max_i\{f_{ij}\}$

Term Weights: Inverse Document Frequency

- **Inverse Document Frequency (IDF)**
  - Terms that appear in many documents are less useful for discrimination

df_i = document frequency of term i

= number of documents containing term i

idf_i = inverse document frequency of term i,

$= \log_2 (N/ df_i)$

(N: total number of documents)

- An indication of a term's *discrimination* power.
- Log used to dampen the effect relative to *tf*.

TF-IDF Weighting

- Combining TF and IDF:

$$w_{ij} = tf_{ij}\, idf_i = tf_{ij} \log_2 (N/ df_i)$$

- High weight when:
  - Term is frequent in the document
  - Term is rare in the collection
- TF–IDF balances:
  - Local importance (TF)
  - Global rarity (IDF)

TF-IDF Example

Given a document containing terms with given frequencies:

A(3), B(2), C(1)

Assume collection contains 10,000 documents and document frequencies of these terms are:

A(50), B(1300), C(250)

Then:

A: tf = 3/3; idf = $\log_2(10000/50)$ = 7.6; tf-idf = 7.6

B: tf = 2/3; idf = $\log_2(10000/1300)$ = 2.9; tf-idf = 2.0

C: tf = 1/3; idf = $\log_2(10000/250)$ = 5.3; tf-idf = 1.8

=>Term A is the most important

Query Representation

Query vector

- The query is treated as a document
- TF–IDF weighting is applied to query terms
- Alternatively, users may specify term weights

Documents and queries are represented in the same vector space

Similarity Measure

- A similarity measure is a function that computes the *degree of similarity* between two vectors.

- Using a similarity measure between the query and each document:
  - It is possible to rank the retrieved documents in the order of presumed relevance.
  - It is possible to enforce a certain threshold so that the size of the retrieved set can be controlled.

Similarity Measure - Inner Product

- Similarity between vectors for the document $\boldsymbol{d}_i$ and query $\boldsymbol{q}$ can be computed as the vector inner product (a.k.a. dot product):

$$\text{sim}(\boldsymbol{d}_j,\boldsymbol{q}) = \boldsymbol{d}_j \bullet \boldsymbol{q} = \sum_{i=1}^{t} w_{ij} w_{iq}$$

where w_{ij} is the weight of term i in document j and w_{iq} is the weight of term i in the query

- Binary vectors:
  - Counts matched terms
- Weighted vectors:
  - Sums products of term weights

Limitations of Inner Product

Problems

- Inner product is **unbounded**
- Favors long documents
- Measures how many terms matched but ignores unmatched terms

=> Not ideal for information retrieval

Inner Product -- Examples

Binary:

	retrieval	database	architecture	computer	text	management	information
– D =	1,	1,	1,	0,	1,	1,	0
– Q =	1,	0 ,	1,	0,	0,	1,	1

Size of vector = size of vocabulary = 7
0 means corresponding term not found in document or query

sim(D, Q) = 3

Weighted:

$D_1 = 2T_1 + 3T_2 + 5T_3 \qquad D_2 = 3T_1 + 7T_2 + 1T_3$

$Q = 0T_1 + 0T_2 + 2T_3$

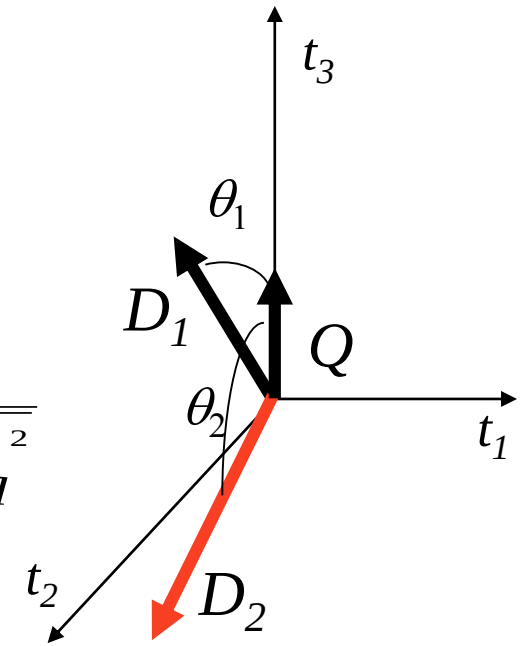
$\text{sim}(D_1, Q) = 2*0 + 3*0 + 5*2 = 10$

$\text{sim}(D_2, Q) = 3*0 + 7*0 + 1*2 = 2$

Cosine Similarity Measure

- Cosine similarity measures the cosine of the angle between two vectors.
- Inner product normalized by the vector lengths.

$$\text{CosSim}(\boldsymbol{d}_j, \boldsymbol{q}) = \frac{\vec{d}_j \cdot \vec{q}}{\left|\vec{d}_j\right| \cdot \left|\vec{q}\right|} = \frac{\sum_{i=1}^{t} (w_{ij} \cdot w_{iq})}{\sqrt{\sum_{i=1}^{t} w_{ij}^{2} \cdot \sum_{i=1}^{t} w_{iq}^{2}}}$$



$D_1 = 2T_1 + 3T_2 + 5T_3$ CosSim(D_1 , Q) = 10 / $\sqrt{(4+9+25)(0+0+4)}$ = 0.81
$D_2 = 3T_1 + 7T_2 + 1T_3$ CosSim(D_2 , Q) = 2 / $\sqrt{(9+49+1)(0+0+4)}$ = 0.13
$Q = 0T_1 + 0T_2 + 2T_3$

D_1 is 6 times better than D_2 using cosine similarity but only 5 times better using inner product.

Ranked Retrieval Process

Naïve Vector Space Retrieval Algorithm

- Convert documents to TF–IDF vectors
- Convert query to a TF–IDF vector
- Compute cosine similarity
- Rank documents by decreasing score
- Return top-ranked documents

Computational Considerations

Time complexity

$$O(|V| \cdot |D|)$$

- $|V|$ vocabulary size
- $|D|$: number of documents

In practice:

- Sparse vectors
- Inverted indexes are used for efficiency

Advantages of the Vector Space Model

- Simple and mathematically well-defined
- Supports partial matching
- Produces ranked results
- Works well in practice
- Basis for modern IR system

Limitations of the Vector Space Model

- No semantic understanding (word sense)
- No syntactic structure (phrases, order)
- Assumes term independence
- Less precise control than Boolean retrieval

- Example:
  - Query “A B” may favor documents with many A but no B

From Vector Space to Modern Models

- **Open Issues and Next Steps**
  - How to model term dependence?
  - How to handle semantics?
  - How to better normalize document length?
- **Leads to:**
  - Probabilistic models
  - BM25
  - Modern neural retrieval models

EXERCISE 1: MANUAL VECTOR SPACE RETRIEVAL

Given the document collection:

DocID	Content
D1	machine learning is fun
D2	deep learning for image recognition
D3	machine learning and data mining
D4	data mining techniques

Query: (machine learning)

Tasks:

- *Build the vocabulary*
- *Compute TF for each term in each document*
- *Compute IDF for each term*
- *Compute TF–IDF weights*
- *Compute cosine similarity between documents and the query*
- *Rank documents by similarity score*

EXERCISE 2: SIMPLE VECTOR SPACE SEARCH

DocID	Nội dung
D1	spam email detection
D2	email security
D3	machine learning for spam
D4	spam filtering techniques

Tasks:

- *Rank documents using cosine similarity*
- *Compare results with Boolean retrieval*

Query: spam email

EXERCISE 3: SIMPLE BOOLEAN SEARCH (PYTHON)

Given the document collection:

DocID	Content
D1	artificial intelligence and machine learning
D2	machine learning applications
D3	artificial intelligence in healthcare
D4	data science and data mining

Query: (artificial AND intelligence) AND NOT healthcare

Tasks:

1.Build the vocabulary

2.Identify documents satisfying the Boolean query

3.Explain why each document is included or excluded

EXERCISE 3: TF–IDF COMPUTATION

Given the document collection:

- information (4), retrieval (2), model (1)

Assume:

- N = 1000 documents
- df(information) = 500
- df(retrieval) = 50
- df(model) = 100

Questions:

- Which term has the highest TF?
- Which term has the highest IDF?
- Which term is most important according to TF–IDF?
- Why is TF alone not sufficient?

EXERCISE 4: COSINE SIMILARITY

Given vectors:

D1 = (2, 3, 5)

D2 = (3, 7, 1)

Q = (0, 0, 2)

- **Tasks:**
- Compute inner product
- Compute cosine similarity
- Compare results

EXERCISE 5: RANKED RETRIEVAL AND USER FEEDBACK

Query: machine learning

Retrieved documents:

Rank	DocID	CosSim
1	D1	0.92
2	D3	0.75
3	D2	0.30

User Feedback:
D1: Relevant
D2: Irrelevant

Questions:

- *How can a Vector Space Model use this feedback?*
- *Why is this difficult in Boolean retrieval?*
- *What information does Vector Space Model have that Boolean does not?*